

Teach Module 40: Application Security Testing

This guide was created from the chat session for Module 40. The course document was used only to identify the module topic. The teaching content below is original practice-oriented instruction.

Module Topic

Application Security Testing

Application security testing is how developers and teams check whether an application has weaknesses before attackers, users, or production traffic expose them.

For a Java/Spring Boot developer, security testing usually answers questions like:

- Can someone bypass authentication?
- Can someone access another user's data?
- Are secrets, tokens, stack traces, or database errors being leaked?
- Is unsafe code present before the app even runs?
- Did a new pull request introduce a known vulnerability?

```
SAST = tests code before it runs
DAST = tests the running app from outside
CI/CD = runs scans automatically
Severity = how bad it is
Priority = how urgently your team should fix it
Remediation = verify, fix, retest, document
```

1. SAST: Static Application Security Testing

SAST analyzes source code, configuration files, dependencies, and build files without running the application.

Think of it like spellcheck for security. It looks for patterns that often indicate vulnerabilities.

Example of risky Java code:

```
String query = "SELECT * FROM users WHERE email = '" + email + "'";
```

This is risky because user input is joined directly into SQL. A SAST tool may flag this as possible SQL injection.

A safer version uses parameters:

```
@Query("select u from User u where u.email = :email")
Optional<User> findByEmail(@Param("email") String email);
```

SAST is good for finding:

- SQL injection patterns
- hardcoded passwords or API keys
- unsafe deserialization
- weak cryptography
- insecure configuration

- dependency vulnerabilities

Common SAST-style tools in Java projects include SonarQube, Semgrep, Checkmarx, Fortify, Snyk Code, and GitHub code scanning.

2. DAST: Dynamic Application Security Testing

DAST tests a running application from the outside.

Think of it like someone probing your deployed app through HTTP requests. It does not need to understand your source code. It behaves more like an attacker or security tester.

For example, if your Spring Boot app exposes this endpoint:

```
GET /users/42
```

A DAST scanner might try requests like:

```
GET /users/42'  
GET /users/42 OR 1=1  
GET /admin  
POST /login with unusual payloads
```

DAST is good for finding:

- broken authentication
- missing authorization checks
- exposed admin routes
- bad HTTP headers
- reflected cross-site scripting
- insecure cookies
- server error leakage
- runtime misconfiguration

Common DAST tools include OWASP ZAP, Burp Suite, Invicti, and AppScan.

3. SAST vs. DAST

SAST asks:

```
Does the code look dangerous?
```

DAST asks:

```
Does the running app behave dangerously?
```

You usually want both. SAST catches issues earlier in the development process, while DAST catches issues that appear only when the app is running.

4. Security Testing in CI/CD

In a real delivery pipeline, security tests often run automatically.

Example pipeline:

```
Developer pushes code
|
Build application
|
Run unit tests
|
Run SAST scan
|
Build container image
|
Run dependency/image scan
|
Deploy to test environment
|
Run DAST scan
|
Approve or block release
```

Not every finding should block a deployment. Teams usually define rules such as:

- Block on critical vulnerabilities.
- Block on high vulnerabilities in production-facing code.
- Warn on medium issues.
- Track low issues as backlog work.

5. Severity Ratings

Security tools usually label findings as:

- **Critical:** likely exploitable, major impact, fix immediately.
- **High:** serious risk, should block release unless an exception is approved.
- **Medium:** real issue, prioritize based on exposure.
- **Low:** minor issue, clean up when practical.
- **Informational:** useful note, not necessarily a vulnerability.

Severity is not the same as priority.

For example, a high issue in an internal admin tool used by three people may be less urgent than a medium issue on a public login page.

Priority depends on:

- Is the endpoint public?
- Does it expose sensitive data?
- Is authentication required?
- Is there a known exploit?
- Is the vulnerable code actually reachable?
- Is this in production or only local development?

6. Common Java/Spring Security Issues

Broken Authorization

Unsafe example:

```
@GetMapping("/orders/{id}")
public Order getOrder(@PathVariable Long id) {
    return orderRepository.findById(id).orElseThrow();
}
```

Problem: any logged-in user might request any order ID.

Safer approach:

```
@GetMapping("/orders/{id}")
public Order getOrder(@PathVariable Long id, Authentication auth) {
    return orderService.findOrderForUser(id, auth.getName());
}
```

The service should verify that the order belongs to the current user.

Leaking Internal Errors

Bad response:

```
org.postgresql.util.PSQLException: relation users_private does not exist
```

This reveals implementation details.

Better response:

```
{
  "message": "Unable to process request"
}
```

Log details internally, but do not expose them to users.

Hardcoded Secrets

Unsafe example:

```
private static final String API_KEY = "sk_live_123456";
```

Safer approach:

```
@Value("${payment.api-key}")
private String apiKey;
```

Secrets should come from environment variables, secret managers, or platform secrets.

Unsafe CORS

Unsafe example:

```
config.addAllowedOrigin("*");  
config.addAllowedMethod("*");  
config.addAllowedHeader("*");
```

This can be dangerous if credentials are involved.

A safer approach is to allow only trusted frontend origins.

Missing Security Headers

Security scanners may report missing headers such as:

```
Strict-Transport-Security  
Content-Security-Policy  
X-Content-Type-Options  
X-Frame-Options
```

These headers help browsers defend against certain classes of attacks.

7. Remediation Workflow

When a scan finds an issue:

1. Confirm whether it is real.
2. Check whether the vulnerable code is reachable.
3. Estimate impact.
4. Fix the root cause.
5. Add a regression test if possible.
6. Re-run the scan.
7. Document exceptions if the team chooses not to fix immediately.

8. Practice Exercises

Exercise 1: Identify SAST Findings

Inspect this Spring Boot controller:

```
@GetMapping("/users/search")  
public List<User> search(@RequestParam String email) {  
    String sql = "SELECT * FROM users WHERE email = '" + email + "'";  
    return jdbcTemplate.query(sql, userRowMapper);  
}
```

Tasks:

- Identify the vulnerability.
- Explain why string concatenation is dangerous.
- Rewrite it using a parameterized query.

- Describe what a SAST tool would likely report.

Expected concept: SQL injection detection through static analysis.

Exercise 2: Fix Hardcoded Secrets

Find this bad configuration:

```
payment.api.key=sk_test_123456789
database.password=admin123
```

Replace it with:

```
payment.api.key=${PAYMENT_API_KEY}
database.password=${DATABASE_PASSWORD}
```

Tasks:

- Explain why hardcoded secrets are risky.
- Move the values to environment variables.
- Reference them safely in Spring Boot.
- Keep only placeholder examples in committed files.

Expected concept: secret scanning and configuration hygiene.

Exercise 3: Broken Authorization Review

Inspect this controller:

```
@GetMapping("/orders/{orderId}")
public Order getOrder(@PathVariable Long orderId) {
    return orderRepository.findById(orderId).orElseThrow();
}
```

Tasks:

- Explain the security problem.
- Add logic so users can only access their own orders.
- Write a test for "user A cannot access user B's order."
- Decide whether this would be caught better by SAST or DAST.

Expected concept: authorization testing and access control validation.

Exercise 4: Run a Dependency Vulnerability Scan

Use a Java project with Maven or Gradle.

Tasks:

- Add a deliberately old dependency version.
- Run a dependency scanner such as OWASP Dependency-Check, Snyk, or GitHub Dependabot.
- Review the vulnerability report.
- Upgrade the dependency.
- Re-run the scan.

Expected concept: known vulnerable dependency detection.

Exercise 5: DAST Scan with OWASP ZAP

Run a local Spring Boot app and scan it with OWASP ZAP.

Tasks:

- Start the app locally.
- Run a baseline scan against `http://localhost:8080`.
- Review alerts.
- Classify alerts as critical, high, medium, low, or informational.
- Pick one issue and propose a fix.

Expected concept: dynamic testing against a running application.

Exercise 6: Security Headers Check

Create a Spring Boot app and inspect response headers.

Tasks:

- Send a request using browser dev tools, Postman, or `curl`.
- Check for `Content-Security-Policy`, `X-Content-Type-Options`, `Strict-Transport-Security`, and `X-Frame-Options`.
- Enable or improve security headers using Spring Security.
- Re-test the response.

Expected concept: HTTP hardening and browser-side protections.

Exercise 7: CI Pipeline Security Gate

Design a simple CI/CD security pipeline.

Tasks:

- Write the pipeline stages in order.
- Decide where SAST should run.
- Decide where DAST should run.
- Decide which severities should block deployment.
- Create a rule like: "Critical and High findings fail the build."

Expected concept: security testing integration into delivery pipelines.

Exercise 8: Triage a Security Report

Pretend you receive these scan results:

```
Critical: Remote code execution in dependency
High: SQL injection in search endpoint
Medium: Missing Content-Security-Policy header
Low: Server version header exposed
Informational: Cookie does not define SameSite
```

Tasks:

- Rank them by fix priority.
- Explain your ranking.
- Decide which issues block release.
- Decide which can go into backlog.
- Write a short remediation plan.

Expected concept: severity versus business priority.

9. Module 40 Lab: Application Security Testing

Lab Goal

Practice finding, understanding, and fixing common security issues in a Java/Spring Boot application.

Lab Scenario

You are given a Spring Boot REST API for users and orders. Your job is to act like an application security tester and developer: scan the app, identify risks, fix them, and explain the remediation.

Part 1: Create a Vulnerable Endpoint

Create this sample controller:

```
@RestController
@RequestMapping("/users")
public class UserController {

    private final JdbcTemplate jdbcTemplate;

    public UserController(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    @GetMapping("/search")
    public List<Map<String, Object>> searchUsers(@RequestParam String email) {
        String sql = "SELECT * FROM users WHERE email = '" + email + "'";
        return jdbcTemplate.queryForList(sql);
    }
}
```

Task: identify the security issue.

Expected answer:

The endpoint is vulnerable to SQL injection because user input is directly concatenated into the SQL query.

Part 2: Fix the Vulnerability

Replace the unsafe query with a parameterized query:

```
@GetMapping("/search")
public List<Map<String, Object>> searchUsers(@RequestParam String email) {
    String sql = "SELECT * FROM users WHERE email = ?";
    return jdbcTemplate.queryForList(sql, email);
}
```

Task: explain why this is safer.

Expected answer:

The database treats the email value as data, not executable SQL. This prevents attackers from changing the structure of the query.

Part 3: Broken Authorization Test

Unsafe order endpoint:

```
@GetMapping("/orders/{orderId}")
public Order getOrder(@PathVariable Long orderId) {
    return orderRepository.findById(orderId)
        .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND));
}
```

Task: explain the problem.

Expected answer:

Any authenticated user could request any order ID. The endpoint checks whether the order exists, but not whether the current user owns it.

Fix idea:

```
@GetMapping("/orders/{orderId}")
public Order getOrder(@PathVariable Long orderId, Authentication authentication) {
    String username = authentication.getName();

    return orderRepository.findByIdAndUsername(orderId, username)
        .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND));
}
```

Part 4: Secret Scanning Practice

Find this bad configuration:

```
payment.api.key=sk_test_123456789
database.password=admin123
```

Replace it with:

```
payment.api.key=${PAYMENT_API_KEY}
database.password=${DATABASE_PASSWORD}
```

Task: explain why secrets should not be committed to source control.

Expected answer:

Secrets in source control can be exposed to developers, logs, backups, forks, or attackers. They should be supplied through environment variables, secret managers, or deployment platform secrets.

Part 5: Security Headers Check

Use `curl` or Postman to inspect response headers:

```
curl -I "http://localhost:8080/users/search?email=test@example.com"
```

Look for security headers such as:

```
X-Content-Type-Options
X-Frame-Options
Content-Security-Policy
Strict-Transport-Security
```

Task: classify missing headers as low, medium, or high risk.

Typical answer:

Missing security headers are usually low or medium depending on the application. They may not be directly exploitable alone, but they reduce browser-side protection.

Part 6: Security Report Triage

Given this report:

```
Critical: Remote code execution in dependency
High: SQL injection in user search
High: Broken authorization on order endpoint
Medium: Missing Content-Security-Policy header
Low: Server version header exposed
```

Create a remediation priority list.

Suggested answer:

1. Critical dependency RCE
2. SQL injection
3. Broken authorization

4. Missing Content-Security-Policy
5. Server version header exposed

Part 7: Final Lab Deliverable

Write a short security testing summary:

Security Testing Summary

SAST Findings:

- SQL injection risk in user search endpoint
- Hardcoded secrets in configuration

DAST Findings:

- Missing security headers
- Possible unauthorized access to order resources

Fixes Applied:

- Replaced string-concatenated SQL with parameterized query
- Moved secrets to environment variables
- Added ownership check to order endpoint
- Reviewed HTTP response headers

Remaining Risk:

- Full authenticated DAST scan should be run before production release

Challenge Task

Add one automated test proving that one user cannot access another user's order.

Example test name:

```
userCannotAccessAnotherUsersOrder()
```

The most important habit from this module is simple: do not just fix the issue. Add a test so the issue does not quietly return later.